



## UNIDADE I

---

# Engenharia de Software Ágil Aplicada

Prof. Me. Edson Moreno

# Introdução

- A Engenharia de Software Ágil Aplicada integra técnica, gestão e inovação para enfrentar os desafios do desenvolvimento de sistemas em um ambiente dinâmico e em constante mudança.
- As metodologias ágeis promovem adaptação rápida, colaboração entre equipes e entregas contínuas de valor, assegurando alinhamento às demandas do mercado. Associadas a normas, modelos de qualidade e práticas de segurança, contribuem para a construção de software confiável, eficiente e sustentável.
- A qualidade e a produtividade constituem pilares desse processo: enquanto a qualidade garante desempenho, confiabilidade e segurança, a produtividade otimiza tempo e recursos.
- O enfoque ágil reforça esse equilíbrio por meio de entregas incrementais, melhoria contínua e uso de ferramentas de integração e versionamento, preparando profissionais para atuar com visão estratégica na gestão e construção de sistemas modernos.

# 1. Qualidade e produtividade de software

- A engenharia de software ágil aplicada surgiu como resposta à complexidade e à demanda por sistemas robustos.
- A transição do desenvolvimento artesanal (pouco estruturado) para métodos sistemáticos e padronizados passou a assegurar qualidade, confiabilidade e previsibilidade nos resultados.
- Atualmente, com abordagens ágeis em ciclos curtos, feedback contínuo e integração frequente, resultou em adaptação rápida às mudanças e entrega de valor constante.
- O foco na qualidade e alta produtividade *versus* excelência técnica aprimorou a Garantia de Qualidade (SQA) na prevenção de defeitos com uso de métricas de monitoramento, mesmo em ambientes velozes, reduzindo desperdícios e custos da qualidade.

## Nesta seção, serão abordados:

- Demanda, qualidade e produtividade
- Importância da qualidade no desenvolvimento ágil
- Fatores, atributos e métricas da qualidade do software

# Demanda, qualidade e produtividade

Conceito de Qualidade: Entrega de valor ao usuário por meio de sistemas eficientes, sem defeitos e que atendam integralmente aos requisitos. Isso se reflete no dilema dos custos:

- Alta Qualidade: Pode exigir maior investimento inicial e tempo.
- Baixa Qualidade: Gera "logística reversa" (retrabalho e manutenção), elevando o custo final do produto devido ao excesso de defeitos.
- Produtividade Eficiente: Capacidade de transformar requisitos em funcionalidades reais com agilidade, sem sacrificar processos essenciais (testes, inspeção e suporte).
- O desafio do fornecimento é equilibrar a demanda do mercado com uma produção sem desperdícios.
- Deve-se evitar a armadilha de reduzir custos imediatos eliminando critérios de qualidade, o que compromete a sustentabilidade do produto. É o verdadeiro “apaga incêndio”.

Aumentar a produtividade significa produzir mais com os mesmos recursos, enquanto a qualidade é garantida pelos requisitos não funcionais que definem as normas e padrões. Juntas, essas dimensões asseguram valor ao cliente e eficiência no processo.

# Importância da qualidade no desenvolvimento ágil

## Abordagem Adaptativa (Scrum, XP, Kanban e outros):

- Foco em entregas contínuas e resposta rápida a mudanças.
- Substituição de fases rígidas (Cascata) por um cronograma baseado em fluxo.

## Otimização e Eficiência (Lean & JIT):

- Inspirado no Just-in-Time: entrega baseada na capacidade e redução de desperdícios.
- Objetivo: Maximização do rendimento e otimização de recursos/tempo.

## Qualidade como Habilitador:

- A excelência técnica não é opcional; é o que sustenta a previsibilidade do projeto.

## Qualidade Integrada:

- Diferentemente do modelo tradicional, o SQA (Garantia de Qualidade) ocorre de forma contínua e não apenas no final.

# Controle da Qualidade: GQS – Garantia de Qualidade de Software

- A Garantia da Qualidade de Software – GQS (Software Quality Assurance – SQA) tem como objetivo assegurar que o software atenda aos requisitos especificados, minimizando defeitos e aumentando a confiabilidade e a satisfação do cliente.
- O conceito de qualidade de software envolve a adequação do produto ao uso pretendido, conformidade com os requisitos do software, e capacidade de produzir resultados consistentes e previsíveis.
- O SQA acompanha todas as fases do ciclo de vida do software, desde a concepção dos requisitos até a manutenção pós-entrega.

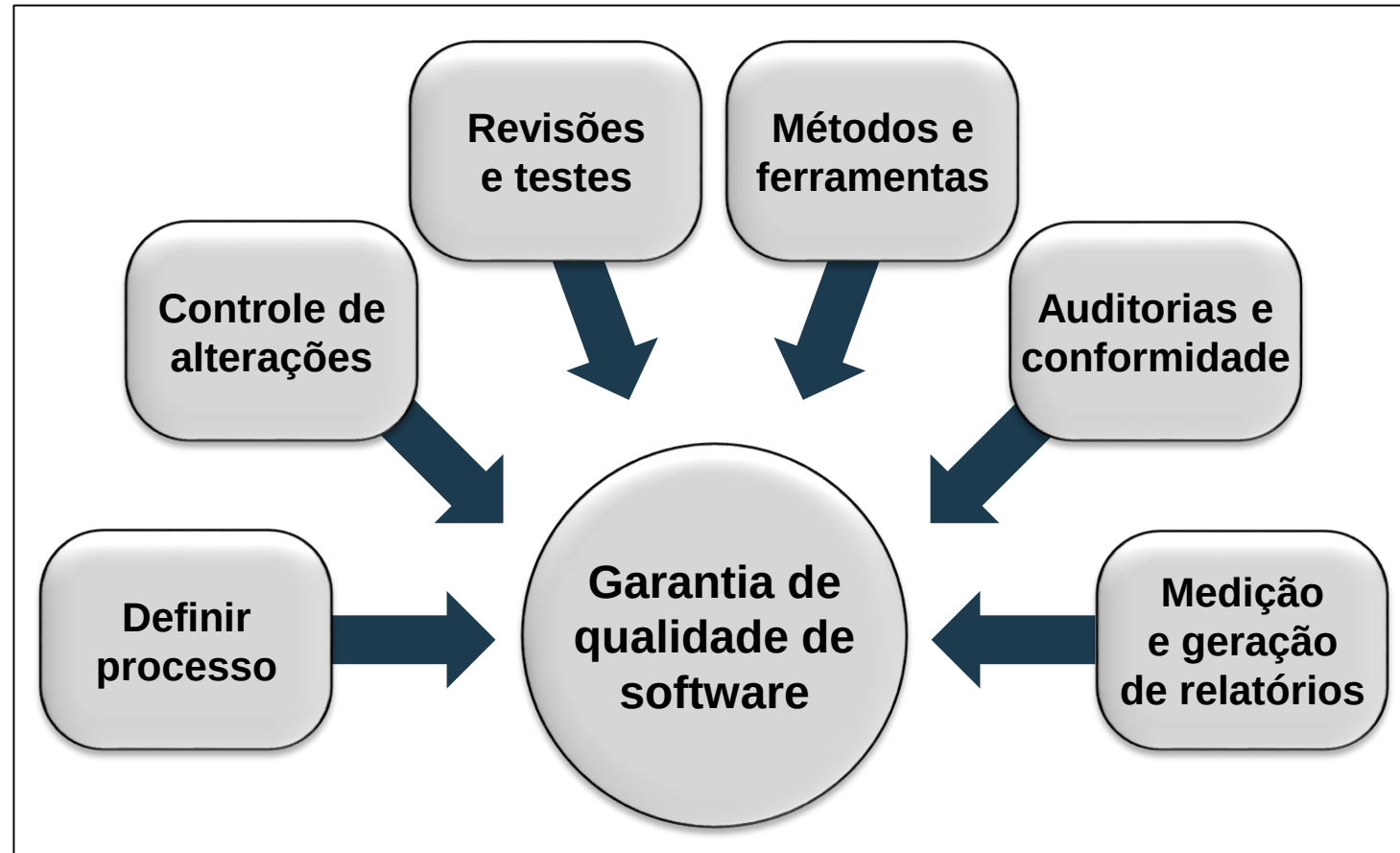


Figura – Garantia de qualidade de software  
Fonte: Pressman (2021,p. 716).

# Práticas eficazes de desenvolvimento de software e a SQA

- Automação de Testes: Reduz o tempo de testes manuais e aumenta a cobertura de testes, melhorando a qualidade sem sacrificar a produtividade.
- Desenvolvimento Ágil: Promove entregas frequentes e iterativas, permitindo ajustes rápidos e contínuos que mantêm a qualidade alta e a produtividade constante.
- Reúso de Código: A reusabilidade é o principal fator de qualidade na engenharia de software.

## Destques da SQA:

- Verificação e Validação (V&V) – envolve testes de software como testes unitários, testes de integração, testes de sistema e testes de aceitação do usuário.
- Análise Estática e Revisão de Pares (Peer Reviews) – trata da inspeção do código por pares.
- Gerenciamento de Configuração de software – tem base no versionamento do software.
- Gestão de artefatos e documentação – são aplicadas rastreabilidade e manutenabilidade do software.
- Gestão do risco – identifica, monitora, avalia e mitiga proativamente os riscos que ameaçam a qualidade e a agenda do projeto.

# Custo da qualidade e metodologias ágeis

- O custo da qualidade (CoQ – Cost of Quality) representa o conjunto de investimentos, esforços e recursos aplicados para garantir que o produto atenda aos requisitos do software, bem como os custos decorrentes da ausência de qualidade.
- Na engenharia de software, o custo abrange todas as atividades voltadas à prevenção, avaliação e correção de falhas ao longo do ciclo de vida do software.
- Nas metodologias ágeis, o custo da qualidade é entendido como parte do valor entregue ao cliente, ou seja, o equilíbrio entre o esforço investido em qualidade e o retorno percebido em confiabilidade, desempenho e satisfação do usuário final.
- Na abordagem ágil, principalmente no Scrum ou no DevOps, a qualidade é continuamente avaliada por meio de iterações curtas, feedback constante, testes automatizados e monitoramento contínuo ao longo de todo o ciclo de desenvolvimento, impactando diretamente no custo do software.



# Custo da qualidade e metodologias ágeis – Conformidades

- O custo da qualidade pode ser entendido como o preço justo que o cliente esteja disposto a pagar. Esses custos podem ser divididos em: custos de conformidade e custos de não conformidades.

Os custos de conformidade são os custos necessários para que o software atenda a padrões e requisitos de qualidade estabelecidos pela equipe desenvolvedora e partes interessadas, como:

- Custos de prevenção – Planejamento, revisões técnicas contínuas, projeto, definição de padrões de código, ferramentas de diagnósticos e testes, e treinamento da equipe de desenvolvimento e dos usuários.
  - Por exemplo: na abordagem ágil: Refinamento de backlog, sessões de pair programming e retrospectivas de sprint.
- Custos de avaliação – São os esforços para inspeções intra e interprocessos, produção, modelos de testes e diagnósticos, precisão, manutenção dos equipamentos e plano de mudanças no software.

# Custo da qualidade e metodologias ágeis – Não conformidades

- Os custos de não conformidades representam os custos associados à falta de qualidade, como falhas, retrabalho ou defeitos que são identificados tardiamente, gerando impacto técnico, financeiro ou de reputação.
- Custos de falha interna – Revisão de código, engenharia reversa, análise do modo de como a falha ocorreu (Root Cause Analysis, tradução para o português como Análise da Causa Raiz), refatoração de código, atrasos na entrega e reparação de defeitos.
  - Exemplo ágil: Falhas identificadas em testes automatizados durante a integração contínua (pipeline de CI – Continuous Integration) exigem correção imediata antes da próxima integração.
- Custos de falha externa – Solução das queixas, devolução e substituição do produto, incidentes de segurança, manutenção da linha de suporte e serviços da garantia de qualidade e índices de satisfação.
  - Exemplo ágil: após o report de um bug por parte de usuários, em fase de implantação, houve custos adicionais para a correção devido a análises de reversão, consequentemente, afetando o fluxo de valor.

# Interatividade

No contexto do desenvolvimento ágil de software, a qualidade assume um papel central para garantir entregas contínuas e previsibilidade. Considerando as abordagens ágeis, os princípios Lean e a integração da qualidade ao processo, assinale a alternativa correta:

- a) A integração contínua da qualidade ao processo ágil permite respostas rápidas às mudanças, reduz desperdícios e sustenta a previsibilidade do projeto por meio da excelência técnica.
- b) A qualidade no desenvolvimento ágil é tratada principalmente na fase final do projeto, por meio de testes de aceitação, garantindo a estabilidade antes da entrega ao cliente.
- c) As metodologias ágeis substituem completamente o planejamento, priorizando apenas a velocidade das entregas em detrimento da qualidade técnica.
- d) O foco principal do Lean e do Just-in-Time no desenvolvimento ágil é aumentar a documentação, garantindo controle formal dos processos.
- e) O modelo ágil mantém a mesma estrutura do modelo em cascata, apenas reduzindo o tempo entre as fases de desenvolvimento.

# Resposta

No contexto do desenvolvimento ágil de software, a qualidade assume um papel central para garantir entregas contínuas e previsibilidade. Considerando as abordagens ágeis, os princípios Lean e a integração da qualidade ao processo, assinale a alternativa correta:

- a) A integração contínua da qualidade ao processo ágil permite respostas rápidas às mudanças, reduz desperdícios e sustenta a previsibilidade do projeto por meio da excelência técnica.
- b) A qualidade no desenvolvimento ágil é tratada principalmente na fase final do projeto, por meio de testes de aceitação, garantindo a estabilidade antes da entrega ao cliente.
- c) As metodologias ágeis substituem completamente o planejamento, priorizando apenas a velocidade das entregas em detrimento da qualidade técnica.
- d) O foco principal do Lean e do Just-in-Time no desenvolvimento ágil é aumentar a documentação, garantindo controle formal dos processos.
- e) O modelo ágil mantém a mesma estrutura do modelo em cascata, apenas reduzindo o tempo entre as fases de desenvolvimento.

# Fatores, atributos e métricas da qualidade do software

- No contexto de qualidade de software, é preciso saber que a qualidade é uma mistura complexa de fatores que variam para cada aplicação e com o que os clientes desejam.

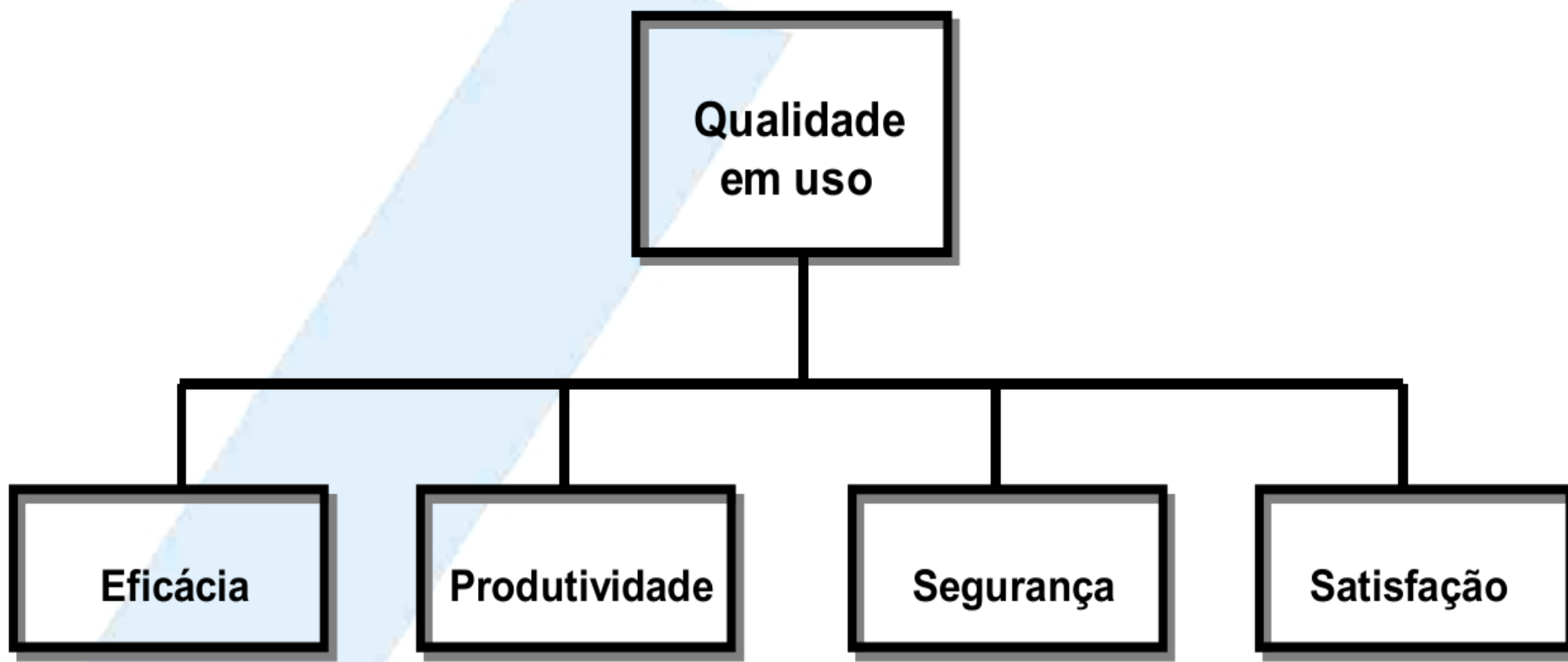
Existem três conceitos distintos para determinar a qualidade:

- Fator de qualidade: O conjunto de fatores de qualidade do produto software permite controlar os níveis dos padrões e requisitos que estão sendo implementados.
- Atributo da qualidade: Representa um aspecto particular do produto, que é determinado em um fator de qualidade. Cada fator de qualidade pode ter vários atributos associados a ele.
- Métricas da qualidade: São medidas quantitativas que permitem qualificar um determinado atributo.

| Algumas métricas de qualidade do software |                                                                            |
|-------------------------------------------|----------------------------------------------------------------------------|
| Métrica                                   | Definição                                                                  |
| Funcionalidade                            | Avaliação da conformidade com os requisitos do cliente.                    |
| Usabilidade                               | Garante a cada categoria de usuário a interface necessária para navegação. |

# Fator de qualidade, atributo e métrica: Como usar

- De acordo com a ISO 9126 (veja na unidade 3 – seção 7.2.1 – NBR ISO/IEC 9126 – Qualidade do produto da engenharia de software), o fator de qualidade usabilidade define quatro características (ou atributos): Eficácia, Produtividade, Segurança e Satisfação.
- Cada um dos atributos vai determinar a forma de medição correta, a métrica, para acompanhar e avaliar o processo.



# Fator de qualidade, atributo e métrica: Mudança de requisitos

## Problemas decorrentes de mudanças nos requisitos:

- Rastreamento do código, evolução do sistema, alteração do código e testes.
- Acoplamento de fatores sem relação direta com a mudança, como o banco de dados com a página web. Mudanças isoladas em componentes do sistema podem afetar todo o sistema.
- Aumento no custo da manutenção.



## **Avaliação de qualidade**

| Métrica              | Definição                                               |
|----------------------|---------------------------------------------------------|
| Acoplamento e Coesão | Grau de acoplamento que pode afetar atributos externos. |
| Manutenibilidade     | Capacidade do produto de software de ser modificado.    |
| Reusabilidade        | Flexibilidade de uso do código.                         |



# Fator de qualidade do software

- O fator de qualidade representa um conjunto de características que influenciam a qualidade do produto de software.
- Cada fator agrupa atributos específicos que devem ser implementados nos componentes do sistema.
- O gerenciamento desses atributos permite controlar padrões e requisitos aceitáveis, garantindo o nível de qualidade do software.

No modelo NBR ISO/IEC 9126, os fatores de qualidade consideram:

- Qualidade interna;
  - Qualidade externa;
  - Qualidade em uso.
- 
- Essas abordagens englobam características como funcionalidade, confiabilidade, usabilidade, entre outras.



# Importância e classificação dos fatores de qualidade

- Os fatores de qualidade ajudam a identificar áreas problemáticas, possibilitando melhorias no produto e no processo de software.
- Contribuem para a evolução contínua da qualidade ao longo do desenvolvimento.

Podem ser classificados em dois grupos:

- Mensuráveis diretamente – podem ser medidos diretamente, no desenvolvimento: por exemplo, mede-se o número de defeitos por ponto de função.
- Mensuráveis indiretamente – podem ser medidos indiretamente, como na usabilidade, que, por exemplo, mede o número de erros cometidos por usuários iniciantes ou satisfação do usuário.

# Atributo da qualidade

- O atributo da qualidade é uma característica específica do produto de software que pode ser medida e avaliada.
- Um ou mais atributos fornecem informações quantitativas e qualitativas para avaliar um fator de qualidade.
- Esses atributos permitem analisar o nível de atendimento aos requisitos funcionais e não funcionais.

## Exemplo (Sistema ERP – Logística):

- Usabilidade: índice de satisfação do usuário e volume de mudanças solicitadas.
- Implementação: cumprimento de prazos e custos, monitorados por cronogramas.
- Integração: grau de compatibilidade e portabilidade com outras tecnologias.
- Atualizações: frequência de mudanças, indicando robustez e estabilidade do sistema.

# Medidas coletadas de atributos da qualidade

| Requisito Funcional (RF)   | Sistema de Informações Logísticas (SIL)         |      |      |      |
|----------------------------|-------------------------------------------------|------|------|------|
|                            | Requisito Não Funcional (RNF) – Funcionalidade. |      |      |      |
|                            | Atributos da qualidade                          |      |      |      |
|                            | USAB                                            | IMPL | INTI | NVAT |
| Cadastros (Distribuidor)   | X                                               | X    | X    | X    |
| Cadastros (Consumidor)     | X                                               |      | X    | X    |
| Comunicação (Distribuidor) | X                                               | X    | X    |      |
| Comunicação (Consumidor)   |                                                 | X    | X    |      |
| Gerenciamento do BD        |                                                 | X    | X    | X    |
| Controle da conta          | X                                               |      |      |      |
| Relatórios (Distribuidor)  | X                                               | X    | X    | X    |
| Relatórios (Consumidor)    | X                                               |      | X    |      |

# Métricas da qualidade

- Métricas são padrões de medição utilizados para quantificar e qualificar resultados de processos, produtos ou serviços de software.
- Cada métrica expressa o valor quantitativo de um atributo, definido por um formato e tipo de dado.
- As medidas possuem dimensões específicas (tempo, tamanho, armazenamento, taxa etc.), que podem ser combinadas para representar resultados mais complexos.
- Os dados coletados são analisados e comparados com históricos anteriores para identificar desvios e oportunidades de melhoria no processo de software.

Algumas das principais métricas da qualidade:

|                |                    |                  |                  |
|----------------|--------------------|------------------|------------------|
| Acurácia       | Flexibilidade      | Operacionalidade | Recuperabilidade |
| Capacidade     | Funcionalidade     | Produtividade    | Reutilização     |
| Eficácia       | Interoperabilidade | Segurança        | Satisfação       |
| Eficiência     | Manutenibilidade   | Portabilidade    | Usabilidade      |
| Escalabilidade | Modularidade       | Rastreabilidade  | Utilização       |

# Interatividade

A qualidade de software é entendida como uma combinação de fatores que variam conforme o tipo de aplicação e as necessidades dos clientes. Considerando os conceitos de fatores, atributos e métricas da qualidade, assinale a alternativa correta:

- a) Atributos da qualidade representam aspectos particulares do produto e podem ser mensurados por meio de métricas associadas a um fator de qualidade.
- b) Cada atributo da qualidade é independente e não possui relação com fatores de qualidade.
- c) Fatores de qualidade são medidas quantitativas utilizadas diretamente para avaliar atributos específicos do software.
- d) Fatores, atributos e métricas da qualidade possuem o mesmo significado e podem ser utilizados de forma indistinta.
- e) Métricas da qualidade correspondem a características abstratas do software, sem necessidade de quantificação objetiva.

## Resposta

A qualidade de software é entendida como uma combinação de fatores que variam conforme o tipo de aplicação e as necessidades dos clientes. Considerando os conceitos de fatores, atributos e métricas da qualidade, assinale a alternativa correta:

- a) Atributos da qualidade representam aspectos particulares do produto e podem ser mensurados por meio de métricas associadas a um fator de qualidade.
- b) Cada atributo da qualidade é independente e não possui relação com fatores de qualidade.
- c) Fatores de qualidade são medidas quantitativas utilizadas diretamente para avaliar atributos específicos do software.
- d) Fatores, atributos e métricas da qualidade possuem o mesmo significado e podem ser utilizados de forma indistinta.
- e) Métricas da qualidade correspondem a características abstratas do software, sem necessidade de quantificação objetiva.

## 2. Engenharia de segurança de software

- Área responsável por planejar, desenvolver e manter sistemas seguros e confiáveis, protegendo dados e funcionalidades contra falhas, vulnerabilidades e ataques.
- Tem como objetivo garantir confidencialidade, integridade, autenticidade e disponibilidade das informações ao longo de todo o ciclo de vida do software.
- Aplica práticas como análise de ameaças, modelagem de riscos, testes de penetração e criptografia.
- Diante do aumento das ameaças cibernéticas, promove o desenvolvimento de sistemas resilientes, assegurando a confiança dos usuários e a qualidade da informação.

### Nesta seção, serão abordados:

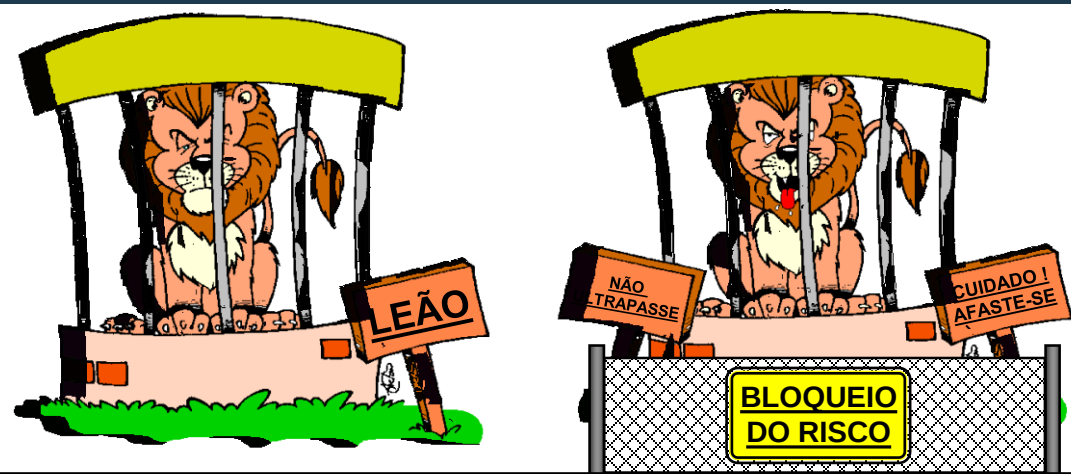
- Gerenciamento e segurança de software
- Gerenciamento de riscos do projeto
- Governança de TI
- Estratégia para suporte de software
- Retorno de investimento (ROI) no ciclo de entrega ágil
- MF&T: melhoria da qualidade de software

# Gerenciamento e segurança de software

- O software é um ativo crítico, cuja falha pode gerar perdas financeiras, danos reputacionais e interrupções de serviço.
- Gerenciamento de Risco, Plano de Contingência e Seguridade (Seguro) visam identificar, avaliar e mitigar riscos, garantindo qualidade, continuidade e confiabilidade dos sistemas.
- Risco é um evento incerto que pode impactar positiva ou negativamente os objetivos organizacionais.
- Análise de Risco é a primeira etapa do gerenciamento da segurança, identificando ameaças e vulnerabilidades técnicas e organizacionais.
  - Utiliza métodos quantitativos e qualitativos para estimar a probabilidade e o impacto dos riscos.



# Elementos de análise do risco na engenharia de software ágil – I



Fonte: Adaptado de:  
<https://www.segurançadotrabalho.ufv.br/wp-content/uploads/2022/06/perigoxrisco.png>

| Elementos     | Definições                                                                                                                                                                                                                                                                                 |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ameaça        | Qualquer evento, pessoa ou condição que possa causar um resultado indesejado, comprometer um artefato ou o processo de software.<br>Exemplo: Atraso no fornecimento de uma API de fornecedor terceirizado.                                                                                 |
| Probabilidade | É a chance de uma ameaça se concretizar durante o desenvolvimento de uma funcionalidade, sprint ou o comprometimento com o ciclo de entregas. Exemplo: há uma grande probabilidade de que a implementação de um novo código contenha regressões, se testes unitários não forem concluídos. |

## Elementos de análise do risco na engenharia de software ágil – II

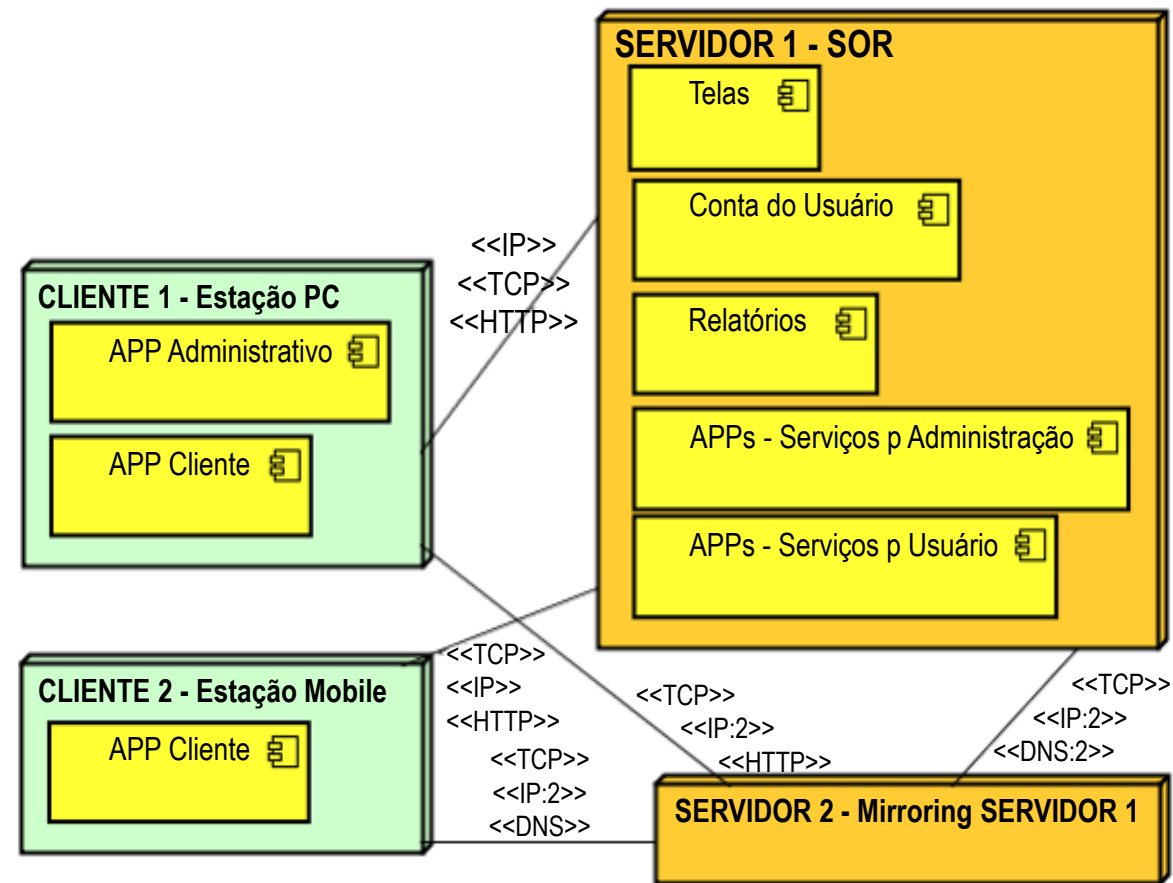
| Elementos               | Definições                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Impacto</b>          | <p>Nível de dano ao software, à equipe ou aos compromissos de entrega (tempo, custo ou qualidade).</p> <p>Exemplo: impacto da mudança, na ocorrência de uma falha de software que impede a implantação do software.</p>                                                                                                                                                |
| <b>Incerteza</b>        | <p>O grau de incerteza é medido de acordo com a falta de informação clara sobre um determinado requisito, uma dependência técnica ou a viabilidade de uma solução. Nas metodologias ágeis, isso é frequente, e é tratado como spike (significado do inglês: atividade de pesquisa ou experimentação técnica), com objetivo de converter a incerteza em informação.</p> |
| <b>Ação Alternativa</b> | <p>É a estratégia de mitigação ou plano de contingência (rollback, que significa ação corretiva) definido para neutralizar ou reduzir o risco. Exemplo: implementar um recurso de fallback (recuo) para serviços de terceiros, ou refatorar o código para reduzir o débito técnico.</p>                                                                                |

# Análise de risco: Sistema de espelhamento (mirroring) para failover

- Arquitetura com espelhamento (mirroring) de servidor para operação de failover (comutação automática para contingência).

## Análise da arquitetura:

- Observe nos estereótipos a mudança de endereços que ocorre de forma automática.
  - O SERVIDOR 1 <<IP>> está à frente do usuário. E, de forma sincronizada, o
  - SERVIDOR 2 – Mirroring <<IP:2>> está sendo atualizado com os dados do SERVIDOR 1 <<IP>>.
- Se o SERVIDOR 1 <<IP>> falhar, ele será desabilitado e, automaticamente, o SERVIDOR 2 – Mirroring <<IP:2>> assume a frente do usuário na rede, até que seja estabilizado o sistema e retornem as operações normais.



# Gerenciamento de riscos do projeto

- O risco pode ser conceituado como a combinação da probabilidade da ocorrência de um evento e de suas consequências sobre os objetivos do projeto.

Em termos quantitativos, o risco pode ser representado pela relação:

- $\text{Risco} = \text{Probabilidade (P)} \times \text{Impacto (I)}$
- Probabilidade (P): Quanto é provável que o risco ocorra (por exemplo: baixa, média ou alta).
- Impacto (I): Quanto são severas ou seriam as consequências, caso o risco ocorra (por exemplo: pequeno, moderado ou crítico).
- No gerenciamento de riscos em engenharia de requisitos, cada tipo de requisito está associado a fontes de risco específicas, pois representam níveis distintos de abstração, validação e impacto no projeto.

# Processos de gerenciamento dos riscos do projeto

Das boas práticas do PMBOK® aplicadas às metodologias ágeis, pode-se destacar os principais processos de gerenciamento dos riscos do projeto:

- Planejar o gerenciamento do risco – Definir como as atividades de risco serão conduzidas no projeto, considerando também a dinâmica das metodologias ágeis.
- Identificar os riscos – Levantar riscos técnicos, de negócio e de processo por meio de reuniões de planejamento, histórias de usuário (user story) e retrospectivas.
- Realizar análise qualitativa e quantitativa dos riscos – Priorizar riscos (baixo, médio e alto), conforme seu impacto (pequeno, moderado ou crítico) nas entregas de curto prazo.
- Planejar as respostas aos riscos – Definir estratégias como spikes técnicos (pesquisa, exploração etc.), automação de testes e pair programming, para mitigar incertezas.
- Implementar respostas aos riscos – Aplicar as ações planejadas de mitigação e contingência durante as iterações. Em um ambiente ágil, isso ocorre durante as sprints, com a equipe ajustando atividades, prioridades e recursos.
- Monitorar os riscos – Adotar em reuniões a prática de revisões contínuas e periódicas sobre os riscos, promovendo aprendizado contínuo e atualização do risk backlog (lista organizada e priorizada de riscos).

# Governança em TI

## Alinhando Tecnologia à Estratégia de Negócio

### O que é?

Um ecossistema composto por:

- Estratégias e Políticas: Regras de uso da tecnologia.
- Estruturas Decisórias: Processos para tomadas de decisão ágeis.

### Qual o objetivo?

- Apoiar e Expandir: Garantir que a TI não seja apenas um suporte, mas um motor para os objetivos estratégicos e operacionais da organização.

### Frameworks de Referência

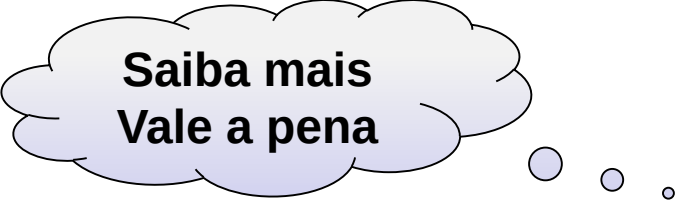
- COBIT® 2019: Foco em Governança e gestão corporativa de TI.
- ITIL® v4: Foco na Gestão de Serviços e entrega de valor.



Fonte:  
<https://img.tfd.com/w/n/1C/664CF-icon.jpg>

# COBIT® 2019: Governança e gestão de TI

- O COBIT® 2019 é a evolução do COBIT® 5, trazendo atualizações significativas, como os fatores de design, que permitem personalizar o sistema de governança de TI conforme o contexto organizacional.
- O COBIT® (Control Objectives for Information and Related Technologies) é um framework de governança e gestão de TI, voltado ao controle de ambientes tecnológicos empresariais.
- O principal objetivo é alinhar as estratégias de TI às estratégias de negócio, maximizando o valor dos investimentos em tecnologia, o gerenciamento de riscos e a conformidade.
- O framework é mantido pela ISACA®, organização internacional sem fins lucrativos, referência global em governança de TI, gestão de riscos, auditoria, segurança da informação e conformidade.



**Saiba mais  
Vale a pena**

ISACA@. Empowering your career. Advancing trust in technology. Site oficial. USA: Illinois, 2026. Disponível em: <https://www.isaca.org/>. Acesso em: 26 jan. 2026.



# ITIL® v4 e governança ágil

<< Da Rigidez à Flexibilidade: Foco total na Entrega de Valor >>

## ITIL® v4: Gestão de Serviços

- Padrão Global – Focado na experiência do cliente e transformação digital.
- Ecosystema Integrado – Une o gerenciamento de serviços a práticas Lean (ou Mentalidade Enxuta), Agile e DevOps.
- Melhoria Contínua: Abordagem dinâmica que conecta negócios, desenvolvedores e gestores.

## Governança Ágil (Lightweight Governance)

- É o equilíbrio entre inovação e conformidade por meio de modelos leves:
- Decisões Descentralizadas: Equipes com autonomia técnica dentro de diretrizes corporativas.
- Alinhamento em Tempo Real: Integração contínua entre áreas de negócio e TI.
- Monitoramento Iterativo: Transparência e gestão de riscos via Scrum, Kanban e DevOps.
- Valor Incremental: Entregas rápidas que geram benefícios tangíveis e mensuráveis.



# Interatividade

Na governança e gestão de TI, frameworks como COBIT® 2019 e ITIL® v4 têm sido integrados a modelos de Governança Ágil (Lightweight Governance) para equilibrar conformidade, geração de valor e inovação. Considerando essas abordagens, assinale a alternativa correta:

- a) A Governança Ágil baseia-se em decisões centralizadas e monitoramento sequencial, semelhantes ao modelo em cascata, para garantir previsibilidade e controle.
- b) A Governança Ágil elimina mecanismos de controle e conformidade, priorizando exclusivamente a autonomia total das equipes de desenvolvimento.
- c) O COBIT® 2019 mantém a mesma estrutura do COBIT® 5, sem mecanismos de adaptação ao contexto organizacional, focando exclusivamente no controle técnico da TI.
- d) O ITIL® v4 é um framework restrito à operação de serviços, desconsiderando práticas ágeis, DevOps e a experiência do cliente.
- e) Os fatores de design do COBIT® 2019 permitem personalizar o sistema de governança de TI, enquanto o ITIL® v4 complementa essa governança ao integrar gestão de serviços, práticas ágeis e melhoria contínua orientada ao valor.

## Resposta

Na governança e gestão de TI, frameworks como COBIT® 2019 e ITIL® v4 têm sido integrados a modelos de Governança Ágil (Lightweight Governance) para equilibrar conformidade, geração de valor e inovação. Considerando essas abordagens, assinale a alternativa correta:

- a) A Governança Ágil baseia-se em decisões centralizadas e monitoramento sequencial, semelhantes ao modelo em cascata, para garantir previsibilidade e controle.
- b) A Governança Ágil elimina mecanismos de controle e conformidade, priorizando exclusivamente a autonomia total das equipes de desenvolvimento.
- c) O COBIT® 2019 mantém a mesma estrutura do COBIT® 5, sem mecanismos de adaptação ao contexto organizacional, focando exclusivamente no controle técnico da TI.
- d) O ITIL® v4 é um framework restrito à operação de serviços, desconsiderando práticas ágeis, DevOps e a experiência do cliente.
- e) Os fatores de design do COBIT® 2019 permitem personalizar o sistema de governança de TI, enquanto o ITIL® v4 complementa essa governança ao integrar gestão de serviços, práticas ágeis e melhoria contínua orientada ao valor.

# Estratégia para suporte de software

- O suporte de software como estratégia é um componente essencial para a continuidade, eficiência e longevidade dos sistemas.
  - Vai além da correção reativa de falhas, envolve a gestão proativa de riscos, melhoria da experiência do usuário (UX) e satisfação do cliente.
- Uma estratégia eficaz de suporte inclui infraestrutura de atendimento planejada, com autoatendimento e níveis de suporte (N1, N2 e N3), conforme a complexidade do sistema.
  - O uso de métricas de desempenho, como tempo de resposta, tempo de resolução e satisfação do cliente, permite monitorar a qualidade do suporte.
- Integrado à engenharia de segurança e às metodologias ágeis, o suporte torna-se proativo e resiliente, reduzindo riscos e fortalecendo a confiabilidade do software.

# Princípios: Integração da engenharia de segurança com metodologias ágeis

| Princípio                                                   | Objetivo                                                                       | Prática                                                                                                                                                                                                |
|-------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Segurança tratada como requisito não funcional (RNF)</b> | Segurança é tratada como parte da qualidade do software.                       | Definir histórias de usuário, desde o início, com critérios de segurança (security stories).                                                                                                           |
| <b>Resiliência e continuidade</b>                           | O sistema deve suportar falhas (bugs) e ataques sem perda crítica.             | Implementar mecanismos de rollback (reverter), redundância de código e dados e logs de auditoria.                                                                                                      |
| <b>Iteratividade e resposta rápida</b>                      | Entregas curtas favorecem correções e reforços de segurança contínuos.         | Aplicar DevSecOps (Desenvolvimento + Segurança + Operações) e pipelines de entrega e implantação contínua (CI/CD - Continuous Delivery / Continuous Deployment) com testes de segurança automatizados. |
| <b>Cultura de responsabilidade compartilhada</b>            | Segurança é de toda a equipe de desenvolvimento, incluindo cliente e usuários. | Pair programming e revisões de código com foco em vulnerabilidades.                                                                                                                                    |

# Retorno de investimento (ROI) no ciclo de entrega ágil

- No desenvolvimento ágil, além do cumprimento de requisitos técnicos, prioriza-se também a entrega contínua de valor de negócio no menor tempo possível.
- O ROI reflete a eficácia do software em suportar os objetivos estratégicos da organização.

O ROI é maximizado pela adoção de práticas ágeis que garantem que o capital investido se traduza rapidamente em resultados mensuráveis como:

- Minimizar custo de refatoração e feedbacks rápidos – Consiste na entrega de sprints em ciclos curtos, permitindo que stakeholders validem as funcionalidades de forma rápida, reduzindo assim o risco de desenvolver um recurso errado, economizando tempo e recursos.
- Aumento da produtividade e eficiência operacional – Um software que atende com precisão e clareza aos requisitos de negócio se traduz diretamente em maior velocidade de adoção, redução de erros operacionais e melhoria da satisfação do cliente e usuário.

# MF&T: Melhoria da qualidade de software

## Sistema MedicSoft e a crise do crescimento

- O sistema MedicSoft é um sistema de gestão hospitalar que centraliza dados de pacientes, prontuários eletrônicos e processos administrativos.

## Caso: Cenário Atual e Desafios (Caso ASSERTI)

- A empresa ASSERTI desenvolve soluções de software para hospitais e clínicas.
- O Problema: O crescimento da base de clientes gerou falhas de integração e lentidão no sistema hospitalar.

## Diagnóstico das últimas 3 entregas:

- Qualidade: Aumento de 25% nos defeitos em produção.
- Eficiência: Apenas 35% de cobertura de testes automatizados e retrabalho crítico em módulos de faturamento.
- Gestão: Dificuldade em mensurar a produtividade real das sprints.

# MF&T: Estratégia de Garantia da Qualidade (GQS)

## Padronização e Rigor Técnico

Para reverter o cenário, a ASSERTI estruturou um plano baseado em quatro pilares:

1. Padrões de Código: Implementação de práticas baseadas em Cleanroom.  
Cleanroom é uma prática de engenharia que foca na prevenção de defeitos por meio de inspeções rigorosas e verificação formal antes da fase de testes.
2. Revisão por Pares (Peer Review): Obrigatoriedade de revisão via GitHub como requisito mandatório no fluxo de integração para validação coletiva do código antes de sua implantação. O GitHub trata do versionamento e garante rastreabilidade do software.
3. Conformidade Normativa: Uso de checklists baseadas na ISO/IEC 25010 com o foco em desempenho, segurança e confiabilidade.
4. Auditorias Cíclicas: Verificações internas de qualidade a cada 3 sprints.



# MF&T: Ferramentas e métricas de eficiência

- O que não se mede, não se gerencia.

| Métrica    | Ferramenta / Desenvolvedor                      | Características                                                                                                                                                                |
|------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Eficiência | Jira / Atlassian                                | Mede o tempo entre criação de uma tarefa e sua conclusão. Eficiência na remoção de defeitos antes da entrega; permite acompanhamento de bugs, tarefas e qualidade do processo. |
| Eficiência | GitHub – modo Kanban / GitHub, Inc. (Microsoft) | Mede o throughput (taxa de conclusão); tempo médio para corrigir falhas; acompanhamento visual do fluxo de trabalho.                                                           |
| Desempenho | GanttProject / Bartłomiej Nawrocki              | Geração de cronogramas, distribuição de responsabilidades e tarefas (ou sprints); checklist; medição de velocidade da sprint (story points entregues).                         |



# MF&T: Gestão do custo da qualidade

Metas a serem atingidas após 6 meses de implementação no sistema MedicSoft

| Categoria                            | Meta da qualidade                                                                                    | Métrica / Medida                            | Tipo de custo         |
|--------------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------|-----------------------|
| Eficiência do processo               | Reduzir em 20% o tempo médio de resolução de tarefas em três sprints.                                | Lead Time                                   | Prevenção e Avaliação |
| Remoção de defeitos antes da entrega | Aumentar em 30% a taxa de correção de defeitos detectados internamente, antes da entrega ao cliente. | Taxa de defeitos removidos antes da entrega | Falhas internas       |
| Produtividade da equipe              | Elevar a velocidade média da sprint em 10%, mantendo a qualidade estável.                            | Story points entregues / Sprint             | Avaliação             |
| Confiabilidade pós-entrega           | Diminuir em 25% as falhas externas reportadas nos dois primeiros meses após a entrega.               | Quantidade de bugs reportados pós-entrega.  | Falhas externas       |

# MF&T: Resultados esperados e ganhos estratégicos

## Resultados obtidos

- Processo: Redução de 20% no Lead Time.
- Qualidade Interna: Aumento de 30% na detecção de bugs antes do cliente receber.
- Produtividade: Elevação de 10% na velocidade média das sprints.
- Satisfação do Cliente: Queda de 25% nas falhas pós-entrega.

## Conclusão:

- A transição para uma cultura de prevenção e melhoria contínua garante não apenas um software melhor, mas maior previsibilidade financeira e confiança do mercado.



# Interatividade

Na estratégia para suporte de software, a integração entre engenharia de segurança e metodologias ágeis contribui para a continuidade, confiabilidade e qualidade do software. Considerando o princípio de que a segurança deve ser tratada como requisito não funcional (RNF), assinale a alternativa correta:

- a) A segurança deve ser abordada exclusivamente na fase de testes finais, após a implementação completa das funcionalidades.
- b) A segurança deve ser garantida apenas por meio de ferramentas automatizadas, dispensando práticas de engenharia e definição de requisitos.
- c) A segurança em ambientes ágeis é incompatível com entregas curtas, pois exige ciclos longos de validação e controle.
- d) O tratamento da segurança como RNF implica considerá-la apenas como responsabilidade da equipe de infraestrutura e operações.
- e) Tratar a segurança como RNF significa incorporá-la como parte da qualidade do software, definindo histórias de usuário com critérios explícitos de segurança desde as fases iniciais do desenvolvimento.

# Resposta

Na estratégia para suporte de software, a integração entre engenharia de segurança e metodologias ágeis contribui para a continuidade, confiabilidade e qualidade do software. Considerando o princípio de que a segurança deve ser tratada como requisito não funcional (RNF), assinale a alternativa correta:

- a) A segurança deve ser abordada exclusivamente na fase de testes finais, após a implementação completa das funcionalidades.
- b) A segurança deve ser garantida apenas por meio de ferramentas automatizadas, dispensando práticas de engenharia e definição de requisitos.
- c) A segurança em ambientes ágeis é incompatível com entregas curtas, pois exige ciclos longos de validação e controle.
- d) O tratamento da segurança como RNF implica considerá-la apenas como responsabilidade da equipe de infraestrutura e operações.
- e) Tratar a segurança como RNF significa incorporá-la como parte da qualidade do software, definindo histórias de usuário com critérios explícitos de segurança desde as fases iniciais do desenvolvimento.

# Referências

- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.
- SBG. Sistema de Gestão ERP Completo. 2015. Disponível em: <http://www.sbg.com.br/>. Acesso em: 22 jul. 2025.
- SOMMERVILLE, Ian. *Software engineering*. 10a. ed. Boston: Pearson Education Limited, 2016.
- SOMMERVILLE, Ian. *Software engineering*. 10a. ed. Boston: Pearson Education Limited, 2021.

**ATÉ A PRÓXIMA!**